

✓ Homework 1

✓ Installing packages

```
!pip install langchain_google_genai
```

✓ Setup your API key

To run the following cell, your API key must be stored in a Colab Secret named `VERTEX_API_KEY`.

1. Look for the key icon on the left panel of your colab.
2. Under `Name`, create `VERTEX_API_KEY`.
3. Copy your key to `Value`.

```
from google.colab import userdata
GEMINI_VERTEX_API_KEY = userdata.get('VERTEX_API_KEY')
```

✓ Downloading receipts.zip

The codes below download and unzip receipts.zip from Google Drive. receipts.zip contains all images from the Fusion folder on BlackBoard.

```
import gdown
file_id = "1oe2FZd3ZT07nrDqjCafNvx108oF8JF"
download_url = f"https://drive.google.com/uc?id={file_id}"
gdown.download(download_url, "receipts.zip", quiet=False)
```

```
Downloading...
From: https://drive.google.com/uc?id=1oe2FZd3ZT07nrDqjCafNvx108oF8JF
To: /content/receipts.zip
100%|██████████████████| 1.61M/1.61M [00:00<00:00, 45.8MB/s]
'receipts.zip'
```

```
!unzip receipts.zip
```

```
Collecting langchain-google-genai
  Downloading langchain_google_genai-0.0.10-py3-none-any.whl.metadata (2.7 kB)
Collecting langchain-core<2.0.0, >=1.2.5 from /usr/local/lib/python3.12/dist-packages (from langchain-google-genai) (1.2.6)
Requirement already satisfied: pydantic<3.0.0, >=2.0.0 in /usr/local/lib/python3.12/dist-packages (from langchain-google-genai) (2.12.3)
Collecting langchain-google-genai<2.0.0, >=1.56.0 from langchain-google-genai (4.12.1)
Requirement already satisfied: anyio<5.0.0, >=4.8.0 in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0, >=1.56.0->langchain-google-genai) (4.12.1)
Collecting google-auth<3.0.0, >=2.47.0 from google-auth[requests]<3.0.0, >=2.47.0->google-genai<2.0.0, >=1.56.0->langchain-google-genai (2.32.4)
Requirement already satisfied: requests<3.0.0, >=2.28.1 in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0, >=1.56.0->langchain-google-genai) (2.32.4)
Requirement already satisfied: httpx<1.0.0, >=0.28.1 in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0, >=1.56.0->langchain-google-genai) (0.28.1)
Requirement already satisfied: tenacity<9.2.0, >=8.2.3 in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0, >=1.56.0->langchain-google-genai) (9.1.2)
Requirement already satisfied: websockets<15.1.0, >=13.0.0 in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0, >=1.56.0->langchain-google-genai) (15.0.1)
Requirement already satisfied: typing-extensions<5.0.0, >=4.11.0 in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0, >=1.56.0->langchain-google-genai) (4.15.0)
Requirement already satisfied: distro<2, >=1.7.0 in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0, >=1.56.0->langchain-google-genai) (1.9.0)
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from google-genai<2.0.0, >=1.56.0->langchain-google-genai) (1.3.1)
We need two functions:
• image_to_base64: convert your image into Base64 encoded string (basically a sequence of 64 characters to make your image easily transferred via API)
• get_image_data_url: takes your image, converting them into base64 string and construct a suitable input for GEMINI api call
```

```
import base64
import mimetypes

# Helper function to read and encode image
def image_to_base64(img_path):
    with open(img_path, "rb") as img_file:
        return base64.b64encode(img_file.read()).decode('utf-8')

# Helper function to encode local file to Base64 Data URL
def get_image_data_url(image_path):
    # Guess the mime type (e.g., image/png, image/jpeg) based on file extension
    mime_type, _ = mimetypes.guess_type(image_path)
    if mime_type is None:
        mime_type = "image/png" # Default fallback

    encoded_string = image_to_base64(image_path)

    # Construct the Data URL
    return f"data:{mime_type};base64,{encoded_string}"
```

```
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash",
    api_key=userdata.get('VERTEX_API_KEY'), # Ensure this key is set in Colab secrets
    temperature=0,
```

```
)
vertexai=True
```

~~google-corab-1.0.0 requires google-auth==2.43.0, but you have google-auth 2.47.0 which is incompatible.~~

Successfully installed filetype-1.2.0 google-auth-2.47.0 google-genai-1.59.0 langchain-google-genai-4.2.0

Display jpg images. Alternatively, open the folder icon on the left pannel to see the images.

```
from IPython.display import HTML, display
import glob, os

image_paths = glob.glob("*.jpg")
image_paths.sort()
html_content = '<div style="display: flex; flex-wrap: wrap; gap: 20px;">'

for path in image_paths:
    b64 = image_to_base64(path)
    filename = os.path.basename(path) # Clean up path to show just the name

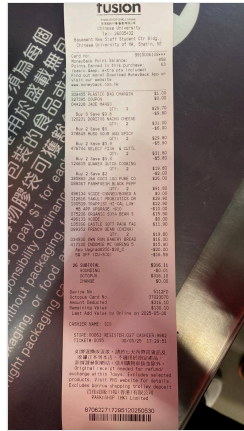
    # Create a vertical column for each image + text
    html_content += f'''
    <div style="display: flex; flex-direction: column; align-items: center;">
        
        <span style="font-family: monospace; font-size: 14px;">{filename}</span>
    </div>
    '''

html_content += '</div>'

display(HTML(html_content))
```



receipt1.jpg



receipt2.jpg



receipt3.jpg



receipt4.jpg



receipt5.jpg



receipt6.jpg



receipt7.jpg

2. Image input to Gemini

Different from text, image needs to be converted into base64 encoded string and then formatted into url before inputting to the language model. It is convenient for image-type input to be transferred through the API.

You can find out more appropriate format for image-type data in this [Link](#)

```
from langchain_core.prompts import ChatPromptTemplate
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("human", [
        {"type": "text", "text": "{question}"},
        {"type": "image_url", "image_url": {"url": "{image_url1}"}}],
    )
```

```
        {"type": "image_url", "image_url": {"url": "{image_url2}"}}},
    ]),

chain = prompt | llm

image_path = "/content/receipt1.jpg"
image_data_url = get_image_data_url(image_path)

image_path2 = "/content/receipt2.jpg"
image_data_url2 = get_image_data_url(image_path2)

response = chain.invoke({
    "question": "What is in this picture?",
    "image_url1": image_data_url,
    "image_url2": image_data_url2
})

print(response.content)
```

This picture contains **two grocery store receipts** from "Fusion by PARKnSHOP limited".

Here's a breakdown of what's on each receipt:

Receipt 1 (Left Side):

```
* **Store:** Fusion by PARKnSHOP
* **Items Purchased:**
*   3 packs of Leek Pork Wontons (20 pcs each), with a discount for deformed packaging on each.
*   2 units of IF100% Coconut Water (with a "Buy 2 Save" discount).
*   3 units of Jade Beans / Edamame (with a "Buy 3 Save" discount).
*   2 units of Fresh Mixed Juice (with a "Buy 2 Save" discount).
*   Nestle Skimmed High Calcium Milk Drink.
*   Mr. Vegetable Packaged Tomatoes (Medium).
*   Mr. Vegetable Zucchini (1 lb).
*   Steamed Pork Patty with Salted Egg.
*   2 units of Golden Imperial Vermicelli.
*   2 units of Frozen Shrimp Meat (Commercial Use).
*   White Willow Mushrooms / Shimeji Mushrooms.
*   Garlic Chicken Chop.
* **Discounts:** Several "Buy X Save Y" discounts, packaging deformation discounts, and a 5% off discount.
* **Total:** $394.70
* **Payment Method:** Octopus card (card number 172731824).
* **Other Info:** Shows a previous online top-up on 2025-04-10 (likely 2024 or 2023, as 2025 is in the future). Cashier name: JL, Cashier ID: 1234.
```

Receipt 2 (Right Side):

```
* **Store:** Fusion by PARKnSHOP limited (Chinese University of HK, Shatin branch).
* **Items Purchased:**
*   Plastic Bag.
*   Coffee (priced at $0.00, possibly a free item or part of a promotion).
*   3 units of Jade Mango (with a "Buy 3 Save" discount).
*   3 units of Doritos Nacho Cheese (with a "Buy 3 Save" discount).
*   2 units of Hugg Sour and Spicy (with a "Buy 2 Save" discount).
*   2 units of Select Fish & Cuttlefish (with a "Buy 2 Save" discount).
*   Quaker Oat Cooking.
```

```
* Japanese Coffee 100% Pure.
* Farmfresh Black Pepper.
* Various "VCODE" items (likely generic codes for specific categories or promotional items).
* Trappist Hi-Cal Low Milk.
* Organic Soya Bean.
* Nestle Soft Pack Facial Tissues.
* French Bean (China).
* Own Run Bakery Bread.
* Indomie Mi Goreng (5 pack).
* **Discounts:** Several "Buy X Save Y" discounts, "MB App Upgrade" discount, "App Coupon", and a 5% off discount.
* **Total:** $316.10
* **Payment Method:** Octopus card (card number 0703070).
* **Other Info:** MoneyBack Point Balance (958 points), Points earned in this purchase (63 points). Transaction date: 30/05/25 17:29:51 (likely 2023 or 2024). Last value added by online on 20
```

In summary, the pictures show detailed records of two separate grocery shopping trips, including the specific items bought, their prices, applied discounts, and payment details via Octopus card

3. Evaluation Code

- Make sure your LLM return a single float as the answer, stored in `query1_answer` and `query2_answer`
- Run the following code blocks: (1) If the blocks does not return any error, then your chain design is correct. Otherwise, please check your chain design.
- Do not modify `query_1_costs` and `query_2_costs`

```
def test_query(answer, ground_truth_costs):
    # Convert string to float if necessary
    if isinstance(answer, str):
        answer = float(answer)

    # Calculate the ground truth sum once for clarity
    expected_total = sum(ground_truth_costs)

    # Check if the answer is within +/- $2 of the expected total
    assert abs(answer - expected_total) <= 2
```

Run the following code block to evaluate query 1:

How much money did I spend in total for these bills?

```
query_1_costs = [394.7, 316.1, 140.8, 514.0, 102.3, 190.8, 315.6] # do not modify this
query1_answer = ...
test_query(query1_answer, query_1_costs)
```

Run the following code block to evaluate query 2:

How much would I have had to pay without the discount?

```
query_2_costs = [480.20, 392.20, 160.10, 590.80, 107.70, 221.20, 396.00] # do not modify this
query2_answer = ...
test_query(query2_answer, query_2_costs)
```

```
[480.20, 392.20, 160.10, 590.80, 107.70, 221.20, 396.00]
```